Multi■Sampler — Essential Features & Implementation Spec

Generated: 2025-09-07 21:22:29 (Asia/Kolkata)

Purpose (one paragraph)

This document defines an essential, input■agnostic Multi■Sampler module: a skeletal framework that can probe a variety of domains (bitmap thumbnails, frameref blobs, GPU textures) using a hybrid sampling strategy (Random + Wallis■Sieve blend), support synchronous and asynchronous plugins, and incorporate temporal■frequency aware sampling. The spec is implementation■oriented with APIs, data formats, algorithms and integration notes.

1. High■level feature list (must■have)

- Input■agnostic adapters (bitmap, frameref blob, GPU texture handle)
- Sampling strategies: Random sampler, Wallis■Sieve sampler (deterministic/low■discrepancy), and composite blending between them
- Temporal frequency weighting (per■pixel / per■region frequency bands and frame history support)
- Plugin API supporting synchronous and asynchronous plugins (topology, prime■ends, SDF, pose, OCR, filters)
- Priority and constraints: per■sample priority, pinned regions, exclusion masks
- Output formats: sample coordinates, sample patches, weighted heatmaps, lookup tables (LUTs), and artifact manifests
- Backpressure and budget controls (maxSamples, CPU/GPU budget, timeBudgetMs)
- Batching & streaming: emit samples in batches, support incremental streaming to consumers
- Deterministic seeding + reproducibility (seeded RNG and deterministic Wallis sequence)
- Metrics & diagnostics: sample counts, timing, queue utilization, quality metrics
- Extensible plugin registry + safe sandboxing (timeouts, cancellations)
- Integration hooks for GPU (export sampler LUT texture) and Storage (artifact keys/manifest)
- Unit tests and deterministic correctness checks for Wallis sieve and random blend

2. Conceptual architecture

Core: MultiSampler class that accepts an InputAdapter and produces SampleBatches. Plugins register transformation/analysis hooks. A Scheduler controls temporal weighting, batching, and budget enforcement. Outputs are delivered to Consumers via callbacks, promises or streams.

3. Data model / types

- InputAdapter: { type: 'bitmap'|'frameref'|'gpu', id, width, height, meta, getPixel(x,y)?, getPatch(x,y,w,h)? }
- SamplePoint: { x: float [0..1], y: float [0..1], weight: float, sourceId, frameNumber?, ts? }
- SamplePatch: { point: SamplePoint, w, h, data: Blob|ImageBitmap|TypedArray, meta }

• SampleBatch: { id, sourceId, method: 'random'|'wallis'|'composite', seed, samples: SamplePoint[], patches?: SamplePatch[], stats }

• PluginResult: any (plugin-defined), but should be serializable for storage

4. API: class MultiSampler (essential methods)

```
class MultiSampler {
  constructor(config = {})

  // Input registration
  attachInput(inputAdapter)           // returns inputHandle
  detachInput(inputHandle)

  // Sampling control
  sampleOnce(inputHandle, options)    // returns Promise<SampleBatch>
  startStream(inputHandle, options)   // starts continuous sampling, returns streamHandle
  stopStream(streamHandle)

  // Plugin management
  registerPlugin(plugin)              // plugin = { id, sync:bool, fn, priority }
  unregisterPlugin(pluginId)

  // Utilities and diagnostics
  setSeed(seed)
  getMetrics()                        // queue utilization, lastSampleTime, throughput
  dispose()
}
```

5. Sampling strategies — detailed

Random sampler (baseline):

Produces uniformly random sample coordinates in the unit image domain. Configurable parameters: sampleCount, stratified (grid-based stratification), jitter, seed. Useful as fallback and for stochastic blending.

Wallis■Sieve sampler (deterministic low■discrepancy):

We define 'Wallis■Sieve' here as a deterministic, reproducible low■discrepancy sampler combining a Vogel (phyllotaxis) spiral with integer ‚sieve' stepping to sample across the domain in rings while skipping with a step sequence. This produces uniform coverage with graceful radial distribution and rapid initial coverage of the domain — suitable for early reads and progressive refinement.

Algorithm (Wallis■Sieve) — steps:

```
function wallisSieveSamples(n, seed=0) {
  // n = desired samples; seed used for angular offset
  // golden angle in radians
  const phi = (1 + Math.sqrt(5)) / 2;
  const ga = 2 * Math.PI * (1 - 1/phi); // golden angle ≈ 2.3999632297
  const samples = [];
  const sievePrimes = [2,3,5,7,11,13];
  let sieveIdx = seed % sievePrimes.length;
  for (let k = 0; k < n; k++) {
    const r = Math.sqrt((k + 0.5) / n);
```

```
    const angle = (k * ga + seed * 0.1) % (2 * Math.PI);
    const x = 0.5 + r * Math.cos(angle);
    const y = 0.5 + r * Math.sin(angle);
    samples.push({ x: clamp(x,0,1), y: clamp(y,0,1), weight: 1.0 });
  }
  return samples;
}
```

Notes: the 'sieve' is implemented by selecting prime-based offsets for angular/radial hops; this name mirrors your project convention. The goal is deterministic, low-discrepancy distribution that is simple to implement and reproducible.

Composite blending: Random + Wallis

```
// options: { randomRatio: 0.3, wallisRatio: 0.7, seed }
function compositeSamples(n, options) {
  const rCount = Math.round(n * options.randomRatio);
  const wCount = n - rCount;
  const rand = randomSampler(rCount, options.seed);
  const wall = wallisSieveSamples(wCount, options.seed);
  const combined = rand.concat(wall);
  return combined;
}
```

6. Temporal-frequency weighting (how to)

MultiSampler must support temporal weighting so that sampling density responds to motion frequency in time. Key ideas: - Maintain short history for each input source: N previous thumbnails or motion scores. - Compute per-region temporal power spectrum (e.g., short-time energy, dominant frequency band). - Map frequency bands → sampling multipliers: e.g., high temporal variance → higher sampling density; low variance → lower density. - Implement two modes: reactive (immediate response to detected motion) and scheduled (periodic re-evaluation to avoid oscillation). Practical API: pass `temporalWindow` and `frequencyBands` in options; sampler returns per-sample `temporalWeight` used to scale selection probability or sample patch size.

7. Plugin API (sync + async)

Plugins allow extending sampling logic (topology, prime-ends, SDF, OCR). A plugin is an object: { id: 'topo-prime', audit: 'bitmap'|'frameref'|'gpu'|'any', sync: true|false, priority: 10, // higher runs earlier fn: (context) => { ... } // if async, returns Promise } Context passed to plugin: { input: inputAdapter, sampleBatch: SampleBatch, // points and optionally patches options: samplerOptions, utils: { fetchArtifact(key), requestPatch(x,y,w,h), cancelToken } } Plugins may modify sample weights, request additional patches for deeper analysis, or annotate samples with plugin results. Constraints: plugins must be sandboxed with timeouts and optional cancellation tokens. MultiSampler enforces a plugin timeBudgetMs per invocation.

8. Integration points (worker, GPU, storage)

- Worker (preprocessor.worker): run lightweight MultiSampler instance on thumbnails (bitmap input). Plugins: fast topology heuristics, phash, region scoring. - Blob worker / Frameref: run heavy MultiSampler instance on frameref blobs (deep topology, SDF). Triggered on-demand via promoteToWork or reprocess commands. - GPU: export sampler result as LUT texture (uSamplerLUT) or binary mask texture for use in reducer shaders (patch_reduce.frag). Provide format: RG channels store x,y indices or offsets; or use a float RGBA EXR LUT for high precision. - Storage: sampler outputs should be written as artifacts (manifest + optional patch blob). Provide keys so downstream consumers can `promoteToWork` or `getReadHandle`.

9. Budgeting, backpressure & graceful degradation

- Configurable budgets: maxSamplesPerBatch, timeBudgetMs, cpuBudgetMs, gpuBudget (texture size), maxConcurrentPlugins. - Adaptive policy: lowWater/highWater thresholds trigger actions: increase maxSamples, reduce patchSize, increase downsampleScale, probabilistic skipping, then dropping as last resort. - Prioritization: frames with `priority` or `hasMotion` are exempted from aggressive thinning. - Observable metrics: queueUtilization, avgProcessingTime, sampleYieldRate.

10. Determinism, testing & unit checks

- Deterministic RNG: use seedable RNG for randomSampler (e.g., xorshift32 or mulberry32). - Reproducible Wallis sequence: ensure sequence depends only on (seed, n). - Unit tests: sample count distributions, coverage metrics, repeatability tests (same seed yields identical samples), plugin timeouts, memory leak checks (ImageBitmap closed). - Visual tests: render heatmaps from sampler output and compare against baselines.

11. Minimal reference implementation sketch (JS pseudo)

```
// constructor options: { seed, maxSamplesPerBatch, randomRatio, downsampleScale, pluginTimeoutMs }
class MultiSampler {
  constructor(opts) { ... }
  attachInput(adapter) { ... }
  async sampleOnce(handle, options={}) {
    const n = options.sampleCount || this.maxSamplesPerBatch;
    const composite = compositeSamples(n, { randomRatio: options.randomRatio||this.randomRatio, seed: 
    // apply temporal weights -> resample or adjust weights
    // run plugins (sync first, then await async with timeout)
    // produce SampleBatch with metadata and stats
    return sampleBatch;
  }
}
```

12. File layout suggestion

- src/js/sampler/MultiSampler.js
- src/js/sampler/adapters/BitmapAdapter.js
- src/js/sampler/adapters/FrameRefAdapter.js
- src/js/sampler/adapters/GpuAdapter.js
- src/js/sampler/plugins/topologyBitmap.js
- src/js/sampler/plugins/wallisSieve.js
- src/js/sampler/test/*.spec.js

13. Acceptance criteria (when is this 'done'?)

- MultiSampler class implemented and exported; basic Random and Wallis samplers pass unit tests. - Plugin API implemented; at least one sync (topology) and one async (phash+SDF request) plugin exist and pass tests. - Worker integration: preprocessor.worker calls sampler on thumbnails and writes sampler manifest artifacts to storage. - GPU integration: sampler can export LUT texture used by a simple reducer shader in demo. - Metrics available and no memory leaks observed after long runs.

Appendix A — References and notes

- Wallis■Sieve here is a project■specific deterministic sampler inspired by phyllotaxis (Vogel) and prime-based stepping to create a 'sieved' spiral sequence. If you prefer classical low-discrepancy sequences (Halton, Sobol), those can be substituted. - Keep sampling implementations small and testable; avoid heavy work inside worker main loop — offload SDF/pose to dedicated blob worker when needed.